arXiv:2610.29680v2 [cs.AI] 22 Oct 2026

TraceForge: Distilling with Your Own Thinking

Yan Wang[1]

[1]Independent Researcher

yanwang.branden@gmail.com

Using an LLM to generate training data for another is among the most general ways to train one, and it remains common practice. Some frontier teams have closed that route for their own models: what comes back is an answer, or a short summary, without the detailed reasoning trace behind it. That is a little ironic: they were themselves built on distilling human knowledge, and distilling an open-source model is still entirely routine; the restriction applies to their own outputs alone. Recent work recovers the hidden reasoning by replaying encrypted blocks across models [1]. It somehow works, but it breaks the providers' terms of service, so it is not something a product can be built on long-term.

We built the alternative. We take the context and the teacher's answer as input, and generate the reasoning trace ourselves, in the style of the student's own model family. We call the result a "family-native" trace. Training is two-stage, supervised and then adversarial: a generator learns to regenerate the family's own reasoning traces from their context and answer, and a critic learns how real a reasoning trace looks. Applied to a teacher's answer, the generator writes one in the family's style. A closed judge scores the family-native traces at 68% against 66% for the real ones, and at 82% once the critic picks the best of four.

We test the idea on five benchmarks with everything else held fixed. On general tasks the five training corpora are about the same; on coding, the generated reasoning beats Fable 5's own thinking content. With the same 3,500 questions and the same teacher answers, HumanEval+ improves by 3.7 points and MBPP+ by 7.1 at the final checkpoint, and the MBPP+ lead holds at every checkpoint. Intuitively, we attribute this to the student and the family-native traces coming from the same model family: the student can absorb them without unlearning its own habits, so adaptation costs less and catastrophic forgetting is smaller.

API[1]

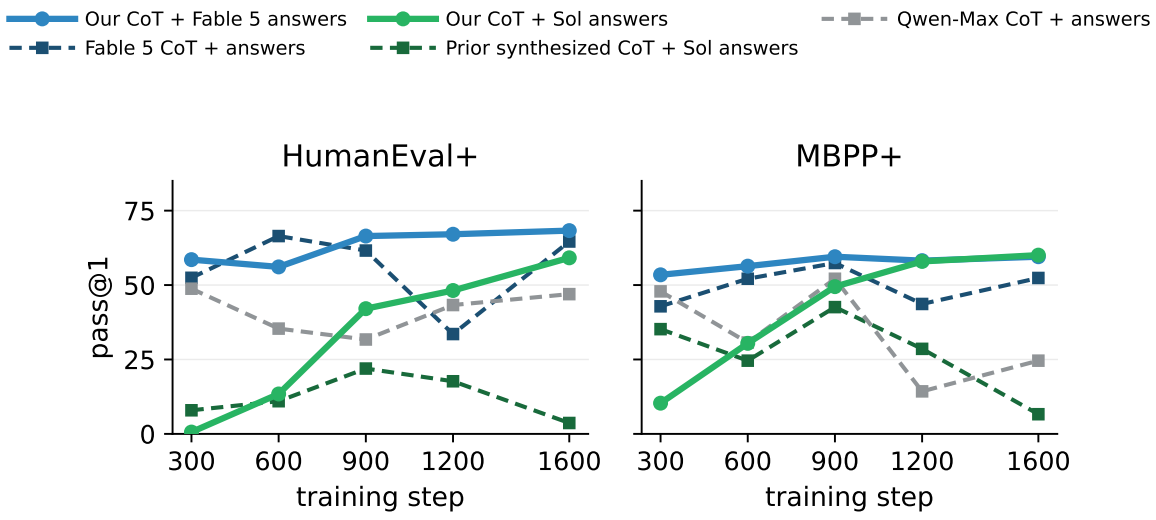

Figure 1 Same answers, better traces. HumanEval+ (left) and MBPP+ (right) across training. Against the model trained on Fable 5's own traces, ours with Fable 5's answers wins at nearly every checkpoint. Both of our models, with Fable 5 and with GPT-6.1-SOL as the teacher, are far better than the one trained with Qwen3.8-Max as teacher.

[1]API supported by ALLE AI Lab. The service currently returns QWEN-family traces only; traces for other model families will follow. Follow the author's RedNote (ID: 9670415842) for updates.

1 Introduction

Using an LLM to generate training data for another is among the most general ways to train one. A strong model answers a set of questions, a weaker one is trained on those answers and, better still, on the reasoning that led to them, and the gap closes [2]. This is not a loophole. Training a model on another model's output is how this field has advanced from the start.

Some labs have decided to stop this. The strongest models now return an answer, or a short summary of their reasoning, and nothing more. The stated reasons are safety and intellectual property; the effect is a moat, and the moat is the point. Others have tried to go around it by hacking the reasoning traces directly: recent work shows that the encrypted reasoning blocks returned by providers are interchangeable across models and can be replayed to recover a hidden reasoning trace verbatim [1]. It works, but it breaks the terms under which those APIs are sold, so it is not something to build on long term.

We propose a legal alternative: do not recover the teacher's reasoning: generate the missing reasoning ourselves. We train a generator to regenerate the reasoning traces of the student's own model family from their context and answer, and a critic to keep those reasoning traces indistinguishable from the family's real ones. Applied to a teacher's answer, the generator writes the reasoning that arrives at it, in the family's style. The two are trained against each other, and at inference the critic picks the best of several candidates, with no labels and nothing that violates a provider's terms. We call the result a family-native trace.

We train Qwen3.5-9B-Base on reasoning traces produced in different ways. On the two coding benchmarks, replacing Fable 5's reasoning trace with a family-native one raises pass@1 by 3.7 and 7.1 points at the final checkpoint, and the MBPP+ lead holds at every checkpoint we measured; on the three general-knowledge tasks, which do not test reasoning, the result is about the same. Reasoning traces synthesized from GPT-6.1-SOL's answers reach a similar level. Family-native traces also beat training on the family's strongest model directly: with Qwen3.8-Max's own reasoning traces and answers, the same student trails our corpus by 21 and 35 points on the same two benchmarks. The teacher's capability sets the ceiling for the student's.

One might be surprised that the generated reasoning traces beat Fable 5's own. We attribute it to the source of the form: a reasoning trace written by the student's own family is data the student can absorb without unlearning its own habits, so adaptation costs less and catastrophic forgetting is smaller than when the same capability arrives in a foreign hand.

Our contribution is easy to state. Training on another model's output is an old and open technique; the only line that matters is whether the data was obtained legally. We use the word distillation without apology, and we stay on the right side of that line. The community can now learn from the best models without asking permission and, on the evidence here, end up with a better student than those models' own reasoning traces would have produced.

2 Method

2.1 Problem

Figure 2 shows the object we work with, a conversation written as a message list. Let R be the reasoning trace and let S, U, A and T be everything else: the system prompt, the user turns, the answer, and any tool call or response. The generator models

$$P(R \mid S, U, A, T),$$

the reasoning trace a model would write given the rest of the exchange.

Training uses conversations from the student's own family only. For the roughly ten thousand reasoning traces we collected from Qwen3.8-Max, all five elements are present, so R is supervised directly and no teacher is involved. At inference we take S, U, A, T from a closed-source model, which supplies everything except the reasoning trace, and sample the reasoning trace the family would have written. The teacher therefore enters

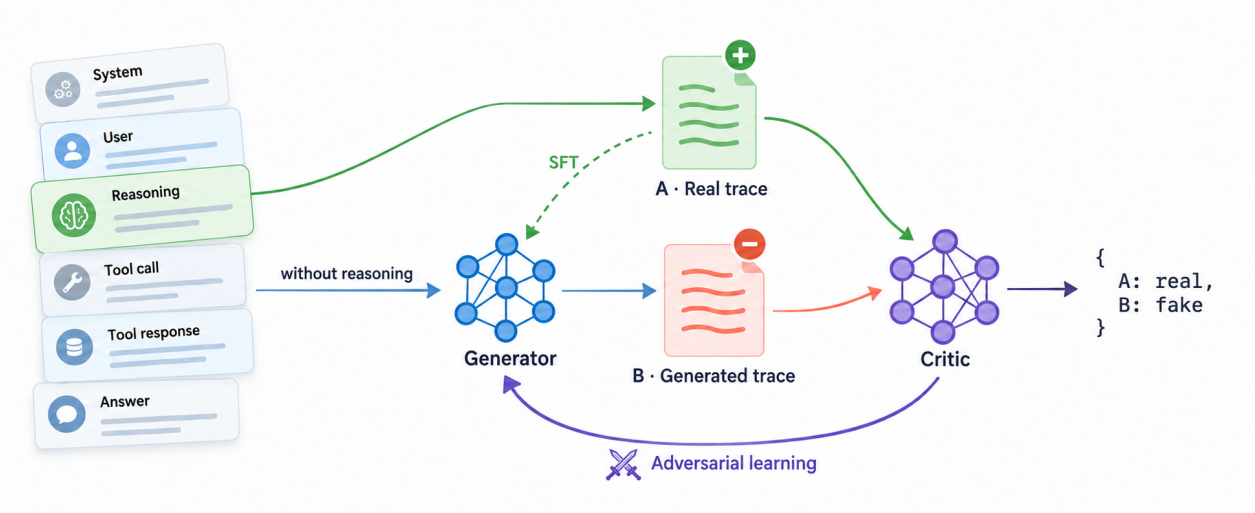

Figure 2 The pipeline. A conversation is a system prompt, user turns, tool calls and responses, an answer, and the reasoning trace that sits before the answer. Training the generator (blue) takes the conversation with the reasoning trace removed and restores it as the target (green). The critic (purple) is shown a real reasoning trace and a generated one and must say which is which. The generator is then trained against the critic's verdict (arc).

only at inference, and only as the source of those four inputs.

2.2 Supervised training of the generator

The generator is a model from the Qwen family. It is fine-tuned on those ten thousand conversations to maximize $\log P(R \mid S, U, A, T)$: the reasoning trace is removed from the input, everything else is kept, and the loss falls on the reasoning trace alone. This is the blue path in Figure 2. Two details carry the design. The target is always a reasoning trace the family itself wrote, so the form is learned from data rather than requested in a prompt. And the answer is part of the input and never part of the output, so the model learns a derivation that *ends* at A rather than a restatement of it.

One consequence is worth stating, because it is easy to misread the result. The reasoning traces we train on were written for the family's own answers. When we later ask the generator for a reasoning trace to accompany a teacher's answer, it cannot be reproducing a reasoning trace it has seen. It is doing what it was trained to do: writing a derivation that ends at that answer, in the family's voice. We call the result a *family-native* trace. No provider gives us the real reasoning trace anyway, and none is needed.

After this stage we ask an outside judge, GPT-6.1-SOL, for the probability that a reasoning trace is real. Real family reasoning traces average 0.66; the generator's average 0.54. That gap is what the rest of the section addresses.

2.3 Supervised training of the critic

The critic is trained to tell real reasoning traces from generated ones, the purple path of Figure 2. For each of the ten thousand samples the generator produced one matched negative, and training stops once the critic assigns the positive a probability at least 0.2 higher than that negative, rather than driving the margin as far as it will go.

We made one mistake here that is worth recording. We assumed that a smaller model could judge a larger one. It cannot: within a few hundred steps the critic fitted shortcuts, scored well on held-out pairs, and gave the generator nothing to learn from. The critic is therefore at least as large as the generator.

2.4 Adversarial co-training

We then run five rounds of online co-training, the arc at the bottom of Figure 2. In each round the generator produces 5,000 reasoning traces, which are shown to the critic alongside the corresponding real reasoning traces. A reasoning trace that fools the critic becomes critic training data; a reasoning trace that does not becomes a preference pair for the generator. At the end of each round we re-score a held-out set with GPT-6.1-SOL.

The score stops improving after five rounds, and we stop there. At that point the generator alone scores 0.68, above the 0.66 of real reasoning traces, and selecting the best of four candidates with the critic scores 0.82.

3 Evaluation

The point of this work is to break the moat of the closed-source model providers, so the test that settles it is a simple one: is a student trained on family-native traces as good as a student trained on the real ones? Everything in this section follows from that question. We hold the base model, the training recipe and the budget fixed, vary only where the reasoning trace and the answer come from, and we simply want to see how the two students compare.

3.1 Setup

Models. Every student is fine-tuned from `Qwen/Qwen3.5-9B-Base`, the pretrained checkpoint, so that the only difference between two students is their training data.

Data. We distil from 5,000 Claude Code trajectories that carry Fable 5's real reasoning traces and answers, at a 256K context window. The source is confidential, but the pool is fixed: every corpus below uses the same prompts and the same Fable 5 answers. From this single pool we build five training corpora that differ only in where the reasoning trace and the answer come from.

- **S1, Fable 5 reasoning + answer.** The original data, used as is: for each of the 5,000 prompts, Fable 5's real reasoning trace together with its own answer.
- **S2, family-native + Fable 5 answer.** As S1, with the reasoning trace replaced by a family-native trace from the generator and critic of Section 2; the answers are unchanged. S1 and S2 therefore differ only in who wrote the reasoning trace.
- **S3, Qwen-Max reasoning + answer.** The prompts of S1 (all context) are given to Qwen3.8-Max, the strongest legal model in the student's own family, and its reasoning traces and answers are used for training.
- **S4, TraceInversion + Sol answer.** We have no GPT-6.1-SOL data carrying real reasoning traces, so we mimic S3: we collect GPT-6.1-SOL's answers on the same prompts and synthesize a reasoning trace for each with TraceInversion (Sol below), the inversion method of Zhang et al. [3].
- **S5, family-native + Sol answer.** As S2, applied to S4: our reasoning traces over GPT-6.1-SOL's answers. S4 and S5 share the answers and differ only in the synthesizer.

Training. Teacher-forced SFT with the prompt masked from the loss. No truncation: every row is under 262,144 tokens, and packed sequences may reach that length. Full parameters, BF16, AdamW with weight decay 0.1 and gradient clipping 1, learning rate 2×10^{-5} under a cosine schedule with 5% warmup down to 2×10^{-6}, micro-batch 1, eight H20 GPUs with TP4/PP2 and sequence parallelism, seed 42, no shuffling. One step is one packed-batch update. The adversarial stage adds five rounds of online preference data, used with the standard DPO objective.

Evaluation. We use three multiple-choice tasks, ARC-Easy, ARC-Challenge and BoolQ, to test general ability in a non-reasoning mode, and two code benchmarks, HumanEval+ and MBPP+, to test coding ability in a

Table 1 Five corpora at the final checkpoint (≈1,600 packed-batch updates). Multiple-choice columns are acc-norm for ARC and accuracy for BoolQ; code columns are pass@1 on the plus suites. S4 is the strongest of the five at step 900 (21.95 and 42.59) and the weakest at the end, which is a collapse rather than a flat line. One run per cell.

Corpus	ARC-E	ARC-C	BoolQ	HumanEval+	MBPP+
S1: Fable 5 reasoning + answer	81.02	58.36	89.63	64.63	52.38
S2: family-native + Fable 5 answer	81.19	59.73	89.33	**68.29**	59.52
S3: Qwen-Max reasoning + answer	81.52	58.28	88.35	46.95	24.60
S4: TraceInversion + Sol answer	81.48	58.53	88.99	3.66	6.61
S5: family-native + Sol answer	80.98	58.19	88.65	59.15	**60.05**

reasoning mode.

3.2 Main comparison

Table 1 gives the final checkpoint. The three knowledge benchmarks do not separate the five corpora: every value sits within 1.6 points of every other, and no setting leads on all three. That is what we expect of tasks that never ask for a reasoning trace, and we set them aside here. The code benchmarks are a different picture, with a spread of more than sixty points.

The first surprise is that S2 beats S1. Replacing the teacher's own reasoning trace with a family-native one adds 3.7 points on HumanEval+ and 7.1 on MBPP+, and Table 2 shows the MBPP+ lead holding at every checkpoint. S1 is the textbook recipe, built on the assumption that the strongest available reasoning trace is the best training signal. On these two benchmarks it is not.

The second surprise is S3. Qwen-Max is a far stronger model than the student, so we expected it to be almost as good a teacher as Fable 5, perhaps a little behind. It is nowhere near: 18 points behind on HumanEval+ and 28 on MBPP+. A capable teacher writing in its own hand transfers much less than a stronger teacher whose answers are re-expressed in the student's hand. Teacher capability turns out to be necessary but not sufficient.

TraceInversion is the weakest of the five, at 3.66 and 6.61, and this one did not surprise us once we read its output. Its reasoning traces are recognisably synthetic: they narrate the solution in a generic voice, use numbered steps the family never writes, and often restate the answer rather than derive it. The score matches what a glance at the data already suggests.

S5 closes the argument with a second teacher. Given exactly the same GPT-6.1-SOL answers as S4, our reasoning traces reach 59.15 and 60.05 against 3.66 and 6.61: 55 and 53 points with nothing varying but the synthesizer. Set beside S2, which shares the synthesizer and differs only in whose answers it writes, S5 is 9 points behind on HumanEval+ and level on MBPP+. That is about the gap one would expect between the two teachers themselves, and it matches the common view of how the two model families compare.

3.3 Further analysis

We also want to know how each corpus behaves over training, not only where it ends. Table 2 reports every intermediate checkpoint on the two code benchmarks, and the trajectories say more than the endpoints do.

S1 is not stable. It reaches 66.46 on HumanEval+ at step 600, falls to 33.54 at step 1,200, and recovers to 64.63 by the end; we re-ran the 1,200-step evaluation twice to rule out a scoring error, and the number held. A 33-point drop that repairs itself is not noise. Our reading is that the teacher's reasoning trace style sits far enough from the family's own that fitting it moves the parameters a long way, and a long move is a move that can overshoot.

S2 and S5 go the other way. S2 rises steadily to 68.29 and 59.52, and S5 rises at every single checkpoint on both benchmarks, from 0.61 and 10.32 to 59.15 and 60.05; on the average of the two, S5 edges past S1.

Table 2 Coding accuracy at every checkpoint. pass@1 on HumanEval+ (upper block) and MBPP+ (lower block); Fig. 1 plots the same numbers.

Step	300	600	900	1,200	1,600
HumanEval+					
S1 teacher reasoning	52.44	**66.46**	61.59	33.54	64.63
S2 family-native	**58.54**	56.10	**66.46**	**67.07**	**68.29**
S3 Qwen-Max reasoning	48.78	35.37	31.71	43.29	46.95
S4 TraceInversion	7.93	10.98	21.95	17.68	3.66
S5 family-native + Sol	0.61	13.41	42.07	48.17	59.15
MBPP+					
S1 teacher reasoning	42.86	52.12	57.41	43.65	52.38
S2 family-native	**53.44**	**56.35**	**59.52**	**58.20**	59.52
S3 Qwen-Max reasoning	47.88	30.69	52.12	14.29	24.60
S4 TraceInversion	35.19	24.60	42.59	28.57	6.61
S5 family-native + Sol	10.32	30.42	49.47	57.94	**60.05**

S5 starts last and finishes near the front. It has the same reasoning traces as S2 and different answers, so the answers are where the difference must come from. Qwen3.5's mid-training is widely said to have used a lot of Claude-family data and very little GPT-family data, which would make GPT-6.1-SOL's answer style new to the base model: it needs steps to adapt to it before the reasoning trace can help. The reasoning trace style is not new, which is why the climb, once it starts, never reverses.

We also have some interesting findings on S3 and S4. Both had their moment and both ended badly: S3 reaches 48.78 on HumanEval+ inside the first 300 steps and never gets back, and S4 peaks at 21.95 and 42.59 at step 900 before falling to 3.66 and 6.61. Reading their outputs, the cause is not subtle. A growing share of the reasoning traces are dead loops or meaningless repetition: a sentence comes back with small variations, or the model keeps re-deriving a result it already has, and the longer the run the more of it there is.

This matches a piece of commonsense among foundation-model teams. Distilling from a distilled model, which people call "breathing second-hand smoke", may result in a student that tends to fall into dead loops while it thinks; in contrast, train on Claude or GPT directly and the problem rarely appears. Intuitively, we guess what happened to S3 and S4 is, in some measure, explained by that one habit. S1, which takes its reasoning traces and answers from Fable 5 directly, never shows the pattern. Finally, we were pleased to find that our family-native traces do not produce it at all.

The two tables together settle the comparison. A family-native trace is not merely a legal substitute for a real one. On the comparison that isolates the reasoning trace, with questions and answers held fixed, **it is even the better teacher**.

4 Discussion and Conclusion

The result is easy to state. Our reasoning traces beat the strongest model we are allowed to distil from, and they beat it by a wide margin: against Qwen-Max's own reasoning traces and answers we are 21 and 35 points ahead. They also edge past the teacher's own reasoning traces, by 3.7 and 7.1 points with the questions and answers held fixed. A comparison from schooling fits. The best student is not produced by the best teacher alone, but by that teacher teaching in the way the student learns best. Our traces are that lesson plan: the strongest teacher's content, delivered in a way that students are familiar with.

Neither the generator nor the critic has ever seen a Claude or an OpenAI token. Every training example comes from Qwen3.8-Max, a model whose terms permit this use, so the pipeline violates no provider's agreement.

One more option, for teams that still see risk in handling Claude or OpenAI answers at all. Generate the family-native trace, hand it to the strongest legally-distillable model in the same family such as Qwen3.8-Max, and let that model write the answer. Then train on the family-native trace and that answer together. No

Claude or OpenAI token is backpropagated at any point, which removes both the legal exposure and the provenance question: **the pipeline yields data that is, on every count, the family's own**. We have run this variant, and it performs essentially the same as S2.

5 Related Work

Learning from reasoning traces. Reasoning traces provide supervision beyond final answers. Distilling Step-by-Step uses teacher-generated rationales as an additional training target for smaller task-specific models [4]; Orca learns from explanation traces generated by larger models [5]; and DeepSeek-R1 transfers reasoning capabilities to smaller models through curated reasoning data [2]. These works motivate training on intermediate reasoning. Our question concerns its source and form: when a teacher's answer is fixed, can a trace synthesized in the student's model-family style teach more effectively than the teacher's original trace?

Recovering and synthesizing hidden reasoning. Panfilov et al. [1] recover protected reasoning by transferring encrypted blocks across models. Synthesis takes a different route: Trace Inversion learns to generate reasoning from inputs, answers and optional summaries [3], while REER works backwards from known-good solutions for open-ended generation [6]. Trace Inversion already shows that synthesized traces can outperform answer-only training and, in some settings, original traces; it also reports weaker coding results associated with style and format mismatch. TraceForge instead trains its generator on the student family's own traces and uses adversarial feedback from a family-trace critic. Our S1/S2 comparison holds Fable 5's answers fixed and replaces only the trace source; S4/S5 holds GPT-6.1-SOL's answers fixed and compares the two synthesis pipelines. These comparisons establish downstream performance differences, but do not isolate family style from every other difference between the pipelines.

Teacher–student distribution differences. MiniLLM changes the distillation objective to reverse KL to better account for the student's approximation of a generative teacher distribution [7]. Our intervention is in the training text: we retain an answer from a strong teacher while synthesizing its accompanying reasoning in a family-native form. This separates the answer source from the trace source. S3, which uses Qwen3.8-Max's own answers and traces, is a distinct baseline; its performance should not be read as an ablation of trace style with answers held fixed.

Selection, verification and rewriting. STaR bootstraps reasoning by retaining successful solutions and using correct answers to guide further rationale generation [8]. Process supervision trains verifiers with step-level correctness labels [9], and self-consistency aggregates sampled reasoning paths by answer agreement [10]. Our critic instead learns to distinguish genuine family traces from generated ones. It supports both generator training and best-of-four selection without step-level correctness labels. Its authenticity score and the independent judge's score measure resemblance to family traces; downstream benchmark performance is evaluated separately.

Adversarial training and the reconstruction target. Self-play fine-tuning contrasts model outputs with human demonstrations [11], and DPO learns from preference pairs [12]. We use a family-trace critic to construct online preference data for the generator. Instruction backtranslation reconstructs an instruction from a response [13]; our reconstruction target is the reasoning trace conditioned on the conversation and answer. Finally, written reasoning need not faithfully reveal the computation that produced an answer [14]. Our goal is a useful family-native training trace, rather than a verbatim reconstruction of the teacher's hidden reasoning.

References

[1] Alexander Panfilov, David Schmotz, Ilia Shumailov, Luca Beurer-Kellner, Joachim Schaeffer, Ameya Prabhu, Jonas Geiping, and Maksym Andriushchenko. Stealing reasoning traces from proprietary llm apis, 2026.

[2] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.

[3] Tingwei Zhang, John X. Morris, and Vitaly Shmatikov. How to steal reasoning without reasoning traces, 2026.

[4] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alex Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017. Association for Computational Linguistics, 2023.

[5] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. Orca: Progressive learning from complex explanation traces of GPT-4, 2023.

[6] Haozhe Wang, Haoran Que, Qixin Xu, Minghao Liu, Wangchunshu Zhou, Jiazhan Feng, Wanjun Zhong, Wei Ye, Tong Yang, Wenhao Huang, Ge Zhang, and Fangzhen Lin. Reverse-engineered reasoning for open-ended generation. In *International Conference on Learning Representations*, 2026.

[7] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Knowledge distillation of large language models, 2023. Original arXiv version introducing MiniLLM.

[8] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022.

[9] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

[10] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022.

[11] Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. Self-play fine-tuning converts weak language models to strong language models, 2024.

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2023.

[13] Xian Li, Ping Yu, Chunting Zhou, Timo Schick, Omer Levy, Luke Zettlemoyer, Jason Weston, and Mike Lewis. Self-alignment with instruction backtranslation, 2023.

[14] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting, 2023.